

Snake Enchanter

Game Design Document – Version 1.8

Author: Julian Gomez | Date: 02.03.2026 | PIP-3 Theme B – SRH Fachschulen

This version reflects the ACTUAL IMPLEMENTED state of the game as of 02.03.2026.

Design deviations from earlier versions are documented in Section 13 (Scope Changes).

1. Project Frame

1.1 Academic Context

Theme: B – Independent 2D or 3D game with a clear core mechanic

Engine: Unity 6 (6000.0.62f1, URP)

Platform: Windows, school laptops, stable 60 FPS, 1920×1080 + ultrawide support

Team: Solo

Duration: 03.02.2026 – 03.03.2026

Implications of Theme B:

- Focus on one clear core mechanic.
 - Deliver a playable, completed prototype with clear win and loss conditions.
 - At least one full gameplay loop: Start → Play → End.
 - Game/round statistics stored in a database via HTTP/JSON API.
 - Structured code and clean project organization.
 - Implement one Elective Content requirement: “Various Modes – min. 2 modes” (Simple & Advanced mode).
-

2. High-Level Concept

2.1 Elevator Pitch

In Snake Enchanter, the player is a wounded warrior trapped in a collapsed ancient ruin, racing against failing health to reach the only escape: a distant window. Unable to fight in their weakened state, the warrior uses discovered magical melodies to command snakes—moving them aside, lulling them into a daze, or conjuring a protective shield. Each melody requires precise timing: hold the tune key to charge a slider, then release when the marker enters the glowing trigger zone. Release too early and the charm fizzles harmlessly; release too late and the snake strikes back. Time is the enemy—health depletes continuously as the warrior weakens.

2.2 Core Fantasy

- The player is a warrior, but not in fighting condition.
- Music becomes the weapon when physical strength fails.
- The ruins are unstable; snakes are both danger and obstacle.
- Every second counts—the warrior is injured and weakening.
- Skillful command—precise slider timing—extends survival; mistakes are costly.

2.3 Target Audience

Age 16+, casual-to-mid-core players, no specific genre experience required.

Should understand goal and controls within 30–60 seconds.

2.4 Project Goals (Theme B Alignment)

- Deliver a clear, understandable core mechanic (slider-based snake command with timing skill).
 - Provide a short, replayable experience (5–8 minutes per run).
 - Implement two modes (Simple, Advanced) from same core level and systems.
 - Integrate database-backed stats to reflect player performance.
 - Support a highly readable, trailer-friendly experience.
-

3. Game Structure

3.1 Game Loop

1. Main Menu → Mode selection (Simple / Advanced).
2. Story Intro screen (atmospheric text, any key to dismiss with fade-in/out).
3. Core gameplay: move through ruin, charm snakes, manage health depletion.
4. Health management: drains passively; restored on successful Move/Daze; snake attacks deal damage.
5. Reach window (win) or health reaches 0 (lose).
6. Win: Ending Story screen → Result Screen with stats.
7. Lose: Immediate Result Screen.
8. Result Screen: shows stats + sends data to backend.
9. Option: Retry (same mode) or return to Main Menu.

3.2 Session Length

Target: 5–8 minutes per run (both modes).

Simple Mode drain: ~0.115 HP/sec → ~4.3 minutes to drain from 30 HP at no HP gain.

Advanced Mode drain: ~0.25 HP/sec → ~2 minutes to drain from 30 HP at no HP gain.

HP restoration from successful tune casts extends survival time significantly.

3.3 Modes (Elective Content – 2 Modes)

Mode 1 – Simple Escape

- All 3 core tunes (Move, Daze, Shield) available from start.
- Passive health drain: ~0.115 HP/sec.
- No additional time pressure.
- Objective: Reach the window with HP > 0.

Mode 2 – Advanced Escape

- All 3 core tunes from start.
 - Passive health drain: ~0.25 HP/sec (approx. 117% of Simple drain rate).
 - Snake attack damage ×1.15 compared to Simple mode.
 - Objective: Reach window with HP > 0.
-

4. Core Gameplay & Mechanics

4.1 Player Abilities

Movement: Free 3D first-person movement using WASD + Mouse.

Camera: First-person view with Cinemachine v3.x (tracks head bone position).

Crouch: Left Ctrl (crouch walk, 50% speed).

Tune Casting: Hold key 1, 2, or 3 → release within trigger zone.

Full body visible: Player's arms and feet visible in first-person view (Pirate Humanoid model).

4.2 Tune System (Core Mechanic) – Genshin-Style Hold & Release Slider

Core Mechanic:

Each tune is mapped to a single key (1, 2, 3 – not Numpad). Player holds the key to charge a slider from 0% to 100%. A colored trigger zone marks the success area. Player must release the key when the slider position is within the trigger zone. Casting requires a snake to be in range.

Slider Behavior:

- HOLD KEY: Slider moves continuously from 0% toward 100%.
- RELEASE KEY: Current slider position is evaluated against trigger zone.
- Slider speed determined by tune duration (shorter duration = faster slider).
- Cooldown applies between casts (same key).

Three Possible Outcomes:

1. TOO EARLY (Release before trigger zone):

- Charm fizzles – no effect.
- Snake returns to idle state.
- No damage to player (safe fail).
- Player can retry immediately.

2. SUCCESS (Release within trigger zone):

- Charm succeeds – snake obeys command (or Shield activates).
- Player health restored for Move/Daze (+X HP). Shield does NOT restore HP.
- Visual/audio confirmation feedback.
- Player spell animation plays.

3. TOO LATE (Release after trigger zone OR slider reaches 100%):

- Charm breaks – loss of control.
- Snake becomes aggressive and attacks.
- Player takes HP damage.

Tune Parameters (IMPLEMENTED):

Tune	Key	Slider Duration	Trigger Zone	Effect
Move	1	3 seconds	40-65%	Snake moves away from player path
Daze	2	4 seconds	35-60%	Snake enters two-phase enchanted state
Shield	3	5 seconds	30-55%	Player shielded from next snake attack (8s)

Note: Tune 4 (Freeze) was removed from v1.0 scope. See Section 13.

4.3 Tunes (Implementation Detail)

Tune 1 – Move Command

- Key: 1
- Slider Duration: 3 seconds.
- Trigger Zone: 40-65%.
- Effect: Targeted snake moves away to a designated MoveAwayTarget position.
- HP Restoration: +X HP on success.
- Active Effects UI: “MOVE” text displays for 3 seconds in corner.
- Player Animation: Spell Casting (Mixamo).

Tune 2 – Daze Command (formerly “Sleep”)

- Key: 2
- Slider Duration: 4 seconds.
- Trigger Zone: 35-60%.
- Effect: Two-phase enchantment — Phase 1 Entranced (3s): amber glow, collision disabled. Phase 2 Dazed (8s): blue glow, Die animation, IsDazed=true.
- HP Restoration: +X HP on success.
- Active Effects UI: “DAZE” text displays for 11 seconds (3s + 8s) in corner.
- Player Animation: Wide Arm Spell Casting (Mixamo).

Tune 3 – Shield Command (formerly “Attack”)

- Key: 3
- Slider Duration: 5 seconds.
- Trigger Zone: 30-55%.
- Effect: Activates ShieldComponent on player. Duration: 8s. Absorbs next snake attack. Breaks on absorb; expires silently.
- HP Restoration: None (Shield is defensive).
- Player Animation: Magic Spell Casting (Mixamo).

REMOVED – Tune 4 (Freeze Command):

- Removed from v1.0 scope. Code base is clean (no Freeze dead code).
- Future Feature: could be re-added in a post-submission update.

4.4 Health & Failure

Health System:

- Continuous HP scale: 0–100.
- Starting HP: 30/100 (player begins injured).
- Display: Health bar UI (left side) with gradient fill (Green → Yellow → Red).
- Pulse animation: activates below 30% HP.

Passive Health Drain:

- Simple Mode: ~0.115 HP/sec.
- Advanced Mode: ~0.25 HP/sec.

Health Restoration (on successful tune cast):

- Move success: +HP.
- Daze success: +HP.
- Shield success: NO HP restoration.
- Cannot exceed 100 HP cap.

Shield System:

- Activated by Tune 3 (Shield) on success.
- Blocks the very next incoming snake attack.
- All snake damage paths route through HealthSystem.TakeSnakeAttackDamage() which checks shield.
- ShieldText TMP visible in Active Effects window while shield is active.

Snake Attack Damage:

- Triggered by: (1) Slider too late / breath attack. (2) Direct collision with active snake.
- Advanced Mode: Damage $\times 1.15$ compared to Simple.
- All damage routed through TakeSnakeAttackDamage() → Shield intercept → TakeDamage().
- Visual feedback: Health bar flash.

Death Condition:

- $HP \leq 0 \rightarrow$ immediate game over.
- Death animation plays based on cause (Drain vs Snake Attack).

- Result screen shown; session data sent to backend.

4.5 Win & Lose Conditions

Win (both modes):

- Reach the window exit trigger with HP > 0.
- Ending Story screen plays → Result Screen with stats.

Lose (both modes):

- Health reaches 0.
 - Result Screen with stats shown immediately.
-

5. World, Level & Progression

5.1 Setting & Tone

Setting: Interior of ancient ruined cave/temple system.

Environment: Stone corridors, broken pillars, fallen debris, cave walls, dim atmosphere.

Tone: Mysterious, slightly tense but not horror. Focus on precision and timing skill.

5.2 Level Structure (IMPLEMENTED)

Single handcrafted level, same for both modes (difficulty via system parameters).

Level uses Cave Parts Set (Unity Asset Store) + Dwarven Pack props (pillars, arches, etc.).

MVP Level Design:

- Entry area with snakes blocking passage.
- Corridor sections requiring tune use.
- Exit: Visible window with light shaft at end of level.
- Single interconnected space (3-area design from original GDD is a future expansion).

Note: The original 3-area design (Tutorial → Escalation → Finale) was reduced to a single level for MVP scope. See Section 13.

6. Controls & UX

6.1 Controls (Keyboard & Mouse)

Action	Input
Movement	WASD
Camera	Mouse move (first-person look)
Crouch	Left Ctrl
Tune 1 – Move	Hold 1, release in trigger zone

Tune 2 – Daze	Hold 2, release in trigger zone
Tune 3 – Shield	Hold 3, release in trigger zone
Dismiss screens	Any key (Story Intro, Ending Story)

6.2 UI Elements

Health Bar:

- Location: Left side of screen.
- Gradient fill: Green (100–50 HP) → Yellow (50–30 HP) → Red (30–0 HP).
- Pulse animation activates below 30% HP.
- No numeric HP display (visual only).

Tune Slider UI (TuneSliderUI v2.2):

- Location: Center-bottom of screen, visible during tune cast only.
- Visual: Segmented horizontal bar, moving marker.
- Yellow = Safe (before zone). Orange = Trigger Zone (success). Gray = Danger (after zone).
- Disappears on key release.

Spell HUD (SpellHUDController v1.1):

- Dynamic slot display per unlocked tune.
- Shows cooldown overlay per slot.
- Range indicator when no snake in range.
- _unlockAllOnStart = false (all tunes visible from start — debug unlock disabled for production).

Active Effects Window (ActiveEffectsController v1.1):

- Displays text labels for active effects in screen corner.
- "MOVE": shows for 3 seconds after successful Move cast.
- "DAZE": shows for 11 seconds (3s Entranced + 8s Dazed) after successful Daze cast.
- "SHIELD": shows while ShieldComponent is active.

Story Intro Screen (StoryIntroController v1.2):

- Atmospheric text displayed before gameplay starts.
- CanvasGroup fade-in (1.2s) → blink hint → any key to dismiss → fade-out (0.6s).
- Input blocked during fade and for 0.8s cooldown after fade-in completes.
- HP drain remains disabled during story intro.

Ending Story Screen (EndingStoryController v1.1):

- Shown on win condition before result screen.
- CanvasGroup fade-in (1.2s) → text → any key to dismiss → fade-out (0.6s).
- Input cooldown 0.8s after fade-in.
- ResultScreenController waits for dismiss before showing win stats.

Result Screen:

- Shows: Mode, Win/Lose, stats summary.
- Sends session data to backend on display.

- Options: Retry same mode / Main Menu.

7. Technical & Data

7.1 Engine & Performance

Unity 6 (6000.0.62f1) with Universal Render Pipeline (URP).

Target: 60 FPS on school laptops, Windows, 1920×1080 + ultrawide.

Input: Unity New Input System (Package) exclusively. Legacy Input FORBIDDEN.

Camera: Cinemachine v3.x — CinemachineBrain on Main Camera, CM_PlayerCamera tracks CameraTarget (empty GameObject under Head Bone in Pirate skeleton).

7.2 Snake AI

NavMesh-based movement via com.unity.ai.navigation v2.0.9.

NavMeshSurface baked on GameLevel scene. NavMeshAgent on each Snake Prefab.

SnakeAI v2.1 State Machine:

- Patrol: Random NavMesh destinations within patrol radius.
- Follow: NavMesh chase toward player when detected.
- MoveAway: NavMesh path to MoveAwayTarget (dedicated GameObject per snake, not child).
- Entranced: Movement stopped, amber glow, collision disabled. Timer 3s → transitions to Dazed.
- Dazed: Blue glow, Die animation, all movement stopped. IsDazed=true prevents idle reset.
- Dead: Snake collapsed from Daze. Remains in collapsed Die animation.
- Breath Attack: Basic Snakes without breath use FollowPlayer instead in breath range.

Snake Types:

- Basic Snake (Toon Snake): Patrol + Follow + Bite attack.
- Advanced Snake (Toon Cobra): Patrol + Follow + Bite + Breath Attack.

7.3 Architecture & Scripts

Namespace	Script	Version	Responsibility
SnakeEnchanter.Core	GameManager.cs	v1.5	Game state, mode, intro/ending flags, two-phase API lifecycle
SnakeEnchanter.Core	GameEvents.cs	—	Static event bus (OnSnakeCharmed, OnHealthChanged, etc.)
SnakeEnchanter.Core	GameState.cs	—	Enum (MainMenu, Playing, Paused, Win, Lose)
SnakeEnchanter.Player	PlayerController.cs	v1.9	WASD + Mouse, Crouch, New Input System, Cinemachine-compatible
SnakeEnchanter.Player	HealthSystem.cs	v1.6	Drain, Events, Death, TakeSnakeAttackDamage() (shield intercept)

SnakeEnchanter.Player	ShieldComponent.cs	v1.2	8s shield, absorbs next snake attack, ShieldText TMP
SnakeEnchanter.Tunes	TuneController.cs	v3.3	3-tune array, slider logic, cooldown, range check, melody trigger
SnakeEnchanter.Tunes	TuneConfig.cs	—	ScriptableObject (key, sliderDuration, triggerZone, effect type)
SnakeEnchanter.Snakes	SnakeAI.cs	v2.1	NavMesh patrol/follow/moveaway/entranced/dazed state machine
SnakeEnchanter.UI	HealthBarUI.cs	v3.1	Gradient, pulse below 30%, CanvasGroup
SnakeEnchanter.UI	TuneSliderUI.cs	v2.2	Segmented bar, moving marker, zone colors
SnakeEnchanter.UI	SpellHUDController.cs	v1.1	Dynamic slots, cooldown overlay, range indicator
SnakeEnchanter.UI	ActiveEffectsController.cs	v1.1	MOVE/DAZE text timers via OnSnakeCharmed
SnakeEnchanter.UI	StoryIntroController.cs	v1.2	CanvasGroup fade, input cooldown, blink hint
SnakeEnchanter.UI	EndingStoryController.cs	v1.1	CanvasGroup fade, input cooldown
SnakeEnchanter.UI	ResultScreenController.cs	v1.2	Waits for ending story dismiss on win; Delete Run button
SnakeEnchanter.UI	MainMenuController.cs	—	Mode selection, scene loading
Audio	MusicManager.cs	v1.1	Scene-based music, gameplay alternation, Recorder-safe
SnakeEnchanter.Data	ApiManager.cs	v1.1	Unity → Backend REST API, fail-silent; LastSessionDbId, PutSession, DeleteSession
SnakeEnchanter.Data	SessionData.cs	—	Session stats struct

7.4 Data & Backend Requirements

Backend: Node.js + Express + MySQL (localhost:3000)

Deployment: START_SERVER.bat for teacher/evaluator setup.

API Endpoints (IMPLEMENTED — all 4 HTTP methods):

Method	Endpoint	Description
POST	/api/game-session	Create pending session at game start
PUT	/api/game-session/:id	Update session with final stats at game end
DELETE	/api/game-session/:id	Remove session from result screen (“don’t save this run”)
GET	/api/leaderboard?mode=...	Top results per mode (simple/advanced)

GET	/api/player-stats	Aggregate stats for result screen display
-----	-------------------	---

Two-phase lifecycle: POST on game start (pending record) → PUT on game end (final stats).

PIP-3 Special Requirement fulfilled: GET + POST + PUT + DELETE all implemented over HTTP/JSON.

Session Data per Run:

sessionId, modeType, success, completionTime, startingHP, endingHP, totalDamageTaken, totalHPRestored, successfulTuneCasts, failedTuneCasts, tooEarlyCount, tooLateCount, snakeBiteCount, timestamp.

8. Assets & Audio

8.1 Visual Assets

Snakes: Toon Snakes Pack (Unity Asset Store, Meshtint Studio).

- Models: Toon Cobra (Advanced), Toon Snake (Basic).
- Animations: Idle, Slither Forward/Left/Right (In Place, no Root Motion), Attack (Bite), Breath Attack, Hit, Die.
- NavMeshAgent: updatePosition=false, manual sync via agent.nextPosition in LateUpdate().
- applyRootMotion = false in SnakeAI script.

Environment:

- Cave system: Caves Parts Set (Unity Asset Store) — Materials on URP/Lit shader.
- Dwarven Pack for props (pillars, arches, window frame exit).
- materialSearch: 2 (Recursive-Up) ensures Unity finds materials in subdirectories.

Player Character:

- Model: Pirate character (FBX, Mixamo Humanoid Rig).
- View: First-person with visible arms and feet (full body model).
- Materials: 8× URP/Lit materials (Body, Cloth, Hair, Weapon, etc.).
- Animations (10 in Animator — Bake Into Pose, Root Motion OFF): Idle, Walk, Crouch Idle, Crouch Walk, Spell Casting (Move), Wide Arm Spell Casting (Daze), Magic Spell Casting (Shield), Death Forward (Drain), Death Left (Snake).
- MC_Controller Parameters: Speed, IsCrouching, SpellMove, SpellDaze, SpellShield, IsDazed.
- Camera: Cinemachine v3.x (CameraTarget: empty GO under Pirate_Skeleton/.../Head bone).

UI Assets:

- Steampunk UI Pack (Gentleland) — frame/border sprites.
- Font: Arvo SDE (TextMeshPro).
- UI Images: teastain textures.
- Controls Overlay: Keycap images for in-game key display.

8.2 Audio

Music (MusicManager v1.1 — scene-based):

- MainMenu: ambient ruins track.
- Gameplay: alternating gameplay music during play.
- Unity Recorder compatible (OnAudioConfigurationChanged debounce fix).

Sound Effects:

- Flute melodies: play on successful tune cast (3 audio clips per tune).
 - Snake sounds: hiss, bite, breath, die.
 - UI sounds: button clicks, result screen.
-

9. Risks & Open Questions

9.1 Design Risks (CURRENT STATUS)

Risk	Status
Teaching cost: Slider mechanic + 3 tunes could overwhelm players.	Mitigated by Story Intro text and Controls Overlay in-game.
Trigger zone balance: Too narrow = frustrating; too wide = trivial.	Values calibrated during playtesting. Final values locked.
"Too Early" being too safe: Players might always release early.	No HP on too-early; must succeed to heal. Mitigation holds.

9.2 Technical Risks (RESOLVED / DEFERRED)

Risk	Status
Slider precision	RESOLVED — Time.deltaTime based slider, tested.
NavMesh Patrol animation	RESOLVED — In Place clips, applyRootMotion=false, LateUpdate sync.
Unity Recorder audio	RESOLVED — Debounce flag in MusicManager v1.1.
HTTP DELETE/UPDATE	RESOLVED — All 4 HTTP methods implemented (v1.8). See Section 7.4.
Spell Scroll in Scene	DEFERRED — SpellUnlockSystem code complete; in-scene pickup not wired for MVP.

10. Balancing Framework

10.1 Core Balance Variables (ACTUAL VALUES)

Health Pool: Maximum HP 100. Starting HP: 30.

Passive Drain Rates (IMPLEMENTED):

- Simple Mode: ~0.115 HP/sec
- Advanced Mode: ~0.25 HP/sec

Health Restoration per successful tune cast:

- Move + Daze: +HP (subject to balancing). Shield: no HP gain.

Snake Attack Damage (IMPLEMENTED):

- Simple Mode: Base damage value.
- Advanced Mode: Base damage $\times$ 1.15.
- Damage applies via TakeSnakeAttackDamage() — Shield intercepts before TakeDamage().

Trigger Zone Sizes (IMPLEMENTED):

Tune	Trigger Zone	Note
Move	40-65%	Same across Simple and Advanced for v1.0
Daze	35-60%	Zone narrowing is a future adjustment
Shield	30-55%	Smallest zone — highest risk/reward

Daze Timing:

- Entranced phase: 3 seconds (amber glow, no collision).
- Dazed phase: 8 seconds (blue glow, Die animation).
- Total effect: 11 seconds.

Shield Duration: 8 seconds (configurable via Tune3_Shield.asset ScriptableObject).

11. Summary for Development

Theme B Requirements:

- ✓ Independent game with a clear core mechanic (slider-based tune control).
- ✓ Complete gameplay loop (Menu → Intro → Play → Story/Result → Retry/Menu).
- ✓ Clear win & lose conditions (window exit or HP $\leq$ 0).
- ✓ Database-backed statistics (POST + PUT + DELETE + 2 $\times$ GET, Node.js + MySQL — all 4 HTTP methods).
- ✓ Structured code & clean project organization (namespaces, SRP, ScriptableObjects).
- ✓ Elective Content fulfilled: 2 modes (Simple vs. Advanced, different drain + damage multipliers).

Remaining for Submission:

- ✓ Windows Build.
 - Game Trailer (Special Requirements — 20 points).
 - ✓ ReadMe for teacher/evaluator.
 - PDF exports of Projektplan + Arbeitsprotokoll.
 - ✓ HTTP DELETE + PUT (UPDATE) endpoints (PIP-3 Special Requirements).
-

12. Version History

Version 1.0 (21.01.2026 – 26.01.2026):

- Initial draft: 3 discrete hearts, no restoration, fixed Advanced timer, 5 areas.

Version 1.1 (27.01.2026):

- Health: 3 hearts → continuous 0-100 HP (starting at 30/100).
- Passive drain + HP restoration on success. Updated UI + database schema.

Version 1.2 (27.01.2026):

- Core Fantasy revised. Tune 3: attacks enemies (not each other). Level: 5→3 areas.
- Balancing Framework (Section 10) added.

Version 1.3 (27.01.2026):

- Tune system: timing-based hold mechanic. Key assignments specified (1–4, not Numpad).
- Tune 2 (Sleep) persistence added. Audio 5–12s melodies.

Version 1.4 (03.02.2026):

- MAJOR: Genshin-Style Hold & Release Slider mechanic.
- Three outcomes: Too Early (safe), Success (+HP), Too Late (damage).
- Slider durations: Move 3s, Sleep 4s, Attack 5s, Freeze 6s.
- Camera: third-person → first-person.

Version 1.5 (09.02.2026):

- Pirate character implemented: Humanoid Rig, 13 animations (movement/spell/death).
- MC_Controller 10 states. TuneController v2.4. HealthSystem v1.3.

Version 1.6 (10.02.2026):

- Snake AI v1.3.x implemented: patrol, proximity detection, 3 attack types.
- NavMesh research completed for v0.3 milestone.

Version 1.7 (02.03.2026):

- SCOPE CHANGES (see Section 13): Tune 3 Attack→Shield, Tune 4 Freeze removed, Sleep renamed to Daze.
- Shield system: ShieldComponent v1.1, blocks next attack, 8s, HealthSystem TakeSnakeAttackDamage().
- NavMesh AI: NavMeshAgent.SetDestination(), In Place animation clips, LateUpdate sync.

- Spell Scroll system: SpellScrollPickup + SpellUnlockSystem (code ready).
- SpellHUDController v1.1: dynamic slots, cooldown overlay, range indicator.
- MusicManager v1.1: scene-based music, Recorder audio debounce fix.
- Story Intro v1.2 + Ending Story v1.1: CanvasGroup fade, input cooldown.
- ActiveEffectsController v1.0: MOVE/DAZE text via OnSnakeCharmed events.
- Advanced Mode: drain ×2.17, Snake damage ×1.15.
- Backend: Node.js + Express + MySQL (POST + 2× GET at this point — PUT + DELETE added in v1.8).

Version 1.8 (02.03.2026 — CURRENT):

- API: All 4 HTTP methods — POST (pending at game start), PUT (final stats at game end), DELETE (result screen), 2× GET.
- Two-phase session lifecycle: ApiManager.LastSessionDbId tracks DB id between POST and PUT/DELETE.
- ResultScreenController v1.2: Delete Run button wired to ApiManager.DeleteSession().
- Script version corrections: TuneSliderUI v2.2, GameManager v1.5, HealthSystem v1.6, ShieldComponent v1.2, ActiveEffectsController v1.1, ApiManager v1.1.
- Section 6.2: _unlockAllOnStart corrected to false (production state, K2 fix from Phase 12).
- PIP-3 Special Requirement: Datenkommunikation über HTTP/JSON — all 4 methods fulfilled.

13. Scope Changes (Deviations from Original GDD)

This section documents intentional deviations from the original design.

Original Design	Actual Implementation	Reason
Tune 3: Attack (snake attacks enemy)	Tune 3: Shield (player protection)	Enemy/creature system too complex for solo MVP
Tune 4: Freeze (all creatures freeze)	Removed from v1.0	Overlapped with Daze; out of scope
Tune 2: Sleep (snake sleeps until passed)	Tune 2: Daze (two-phase: Entranced → Dazed + Die)	More dynamic, better visual feedback
3-Area level (Tutorial → Escalation → Finale)	Single area level	Time constraint; single area fully playable
Advanced Mode: Scroll unlocks Tune 4	Not implemented	Tune 4 removed from scope
Trigger zones wider in Simple mode	Same zones both modes (v1.0)	Simplification; balancing via drain rate instead
HTTP DELETE + UPDATE endpoints	Implemented in v1.8	POST (start) + PUT (end) + DELETE (result screen) + 2× GET
Spell Charges (Advanced mode)	Removed (TuneController v3.3)	Added complexity without core gameplay value
Dynamic collapse timer (Advanced)	Not implemented	Drain rate difference sufficient for difficulty

